

FPGA-Based Low-Power Systolic-Array CNN Accelerator Design

Executive Summary

Convolutional neural networks (CNNs) enable state-of-the-art image classification but demand immense computation. FPGAs offer a compelling platform for CNN inference, providing reconfigurability and parallelism for high throughput at low energy ¹ ². Systolic-array architectures – grids of multiply-accumulate (MAC) processing elements (PEs) – are widely adopted to maximize on-chip data reuse and minimize off-chip memory transfers. This report surveys recent (2020–2025) FPGA CNN accelerator designs, focusing on energy-saving techniques (quantization, pruning, clock gating, DVFS, etc.) and dataflow variants (weight/output/input-stationary, Triangular Input Movement (TrIM) ³, etc.). We discuss FPGA-specific factors (Xilinx/Intel toolchains, DSP/BRAM/URAM resources, floorplanning, clocking, static vs. dynamic power), and compile low-power strategies at algorithmic (quant/prune), RTL (clock/operand gating), and physical (voltage/frequency scaling, memory hierarchy) levels. A systematic design-space exploration (DSE) methodology is outlined, with key metrics (throughput, latency, TOPS/W, energy per inference, accuracy loss). We propose a low-power weight-stationary systolic architecture with triangular dataflow, block diagrams (Mermaid) and detailed PE/memory organization, and an implementation plan (uncommitted device, Vivado/Quartus flows, verification, and power measurement). Benchmarking on CIFAR-10/100 and ImageNet (or Tiny-ImageNet) with INT8 quantization (with retraining) is recommended. Expected results include GOPS/W and energy-per-inference improvements (e.g. tens of GOPS/W with <2% accuracy loss ⁴ ⁵). Tables compare representative designs, FPGA resources, and power techniques. Finally, risks, limitations, and future research directions (sparsity, heterogeneous integration, newer FPGAs) are discussed, and a proposed Gantt-chart timeline is provided.

Background and Motivation

CNNs (e.g. AlexNet, ResNet, VGG) dominate image classification but involve billions of MAC ops per inference. CPUs struggle with such workloads, while GPUs deliver raw throughput at high power. FPGAs, with thousands of DSPs and flexible on-chip memories, enable customized datapaths for CNNs. They achieve **low latency and high energy efficiency** compared to CPUs, and greater flexibility/development speed than ASICs ². For example, a 2022 Xilinx FPGA design reached 115.2 GOP/s on VGG-16 at only 3.8 W (30.3 GOP/s/W) ⁵. However, modern CNNs also strain memory bandwidth and power budgets. Techniques like on-chip data reuse, low-precision arithmetic, and power management are essential to fit CNNs on edge FPGAs ² ⁶.

Systolic arrays form a natural mapping for convolution, streaming data through a PE grid with localized accumulation. Variants include weight-stationary, output-stationary, input-stationary flows ⁷ ³. Recent literature has explored novel dataflows (e.g. Triangular Input Movement) to cut memory accesses and boost energy efficiency ³. In parallel, low-power techniques from algorithmic (quantization, pruning) to circuit-level (clock gating, DVFS) have been applied to CNN inference ⁶ ⁸. Our goal is to survey these advances and synthesize a low-power systolic-array CNN accelerator design for FPGAs.

State-of-the-Art Literature Survey

Systolic-Array CNN Accelerators

Systolic arrays exploit spatial parallelism by arranging PEs in a grid, passing data rhythmically between neighbors. Early work (e.g. Wei *et al.*, DAC 2017) synthesized a parameterized systolic array on FPGAs, achieving up to 1.2 TOPS (8–16 bit) ⁹. They clocked a Virtex device at high frequency (461 GFLOPS FP32; 1.2 TOPS fixed-point) by heavy DSP/LUT usage ⁹. However, such designs often trade area for speed. Later designs have refined dataflows: for instance, Sestito *et al.* (2024) introduced **Triangular Input Movement (TrIM)** – a variant where input activations sweep through the array in triangular waves, keeping weights static in PEs. TrIM cuts memory access by $\sim 10\times$ vs. conventional row-stationary flows, yielding 453.6 GOPS (for VGG16/AlexNet) and $\sim 11.9\times$ better energy efficiency than prior FPGA accelerators ³.

Table 1 summarizes key recent FPGA systolic CNN accelerators:

Reference	Year	Architecture	Device	Dataflow	Precision	Peak Throughput	Energy Efficiency
Wei <i>et al.</i> (DAC) ⁹	2017	Systolic Array, HLS	Xilinx FPGA (Virtex7)	Weight-stationary	FP32/INT8	461 GFLOPS (FP32), 1.2 TOPS (8–16b)	–
Xiang <i>et al.</i> (JSA) ¹⁰	2025	Sparse Systolic (MCBS)	Xilinx/Zynq	Weight-stationary	4/8-bit + sparsity	–	5.36× DSP eff, 8.87× energy eff† ¹⁰
Du <i>et al.</i> (Appl. Sci.) ⁵	2021	Systolic Array (384 PEs)	ZC706 (Zynq-7020)	Configurable systolic	INT8	115.2 GOPS (VGG16)	30.32 GOPS/W ⁵
Sestito <i>et al.</i> (TCSI) ³	2024	Triangular SA	FPGA (unspec.)	TrIM (input-stationary)	INT8	453.6 GOPS (VGG16)	$\sim 11.9\times$ efficiency gain ³
Gundrapally <i>et al.</i> (Electr.) ⁸	2024	Tensil SA (PYNQ)	PYNQ-Z2 (Zynq-7020)	–	INT8 (ResNet-20)	–	$\sim 22\%$ power reduction (ResNet-20) ⁸
Montgomerie-Corcoran <i>et al.</i> (ICFPT) ⁶	2019	DSE Tool (fpgaConvNet)	Virtex (Xilinx)	–	–	AlexNet: (opt)	20.1% power ↓ (via DSE) ¹¹

† Compared to a baseline FPGA design.

The recent trend is toward hybrid/mixed-precision and sparsity-aware SA designs. For example, Hu *et al.* design a bit-level sparse/mixed-bit SA (MCBS) that packs 4/8-bit weights to leverage DSPs, achieving $\sim 5.4\times$ DSP efficiency and $\sim 8.9\times$ energy efficiency improvements ¹⁰. Many of these architectures run binary or ternary networks (FINN streamers ⁷) or use automated toolflows (FINN-R, fpgaConvNet) to unroll loops and generate pipelines ⁷ ⁶.

Quantization and Pruning

Reducing operand precision or pruning weights are primary algorithmic techniques for low-power CNNs. Quantization (e.g. 8-bit INT or mixed 4/8-bit) shrinks model size and MAC complexity. Post-Training Quantization (PTQ) can often be applied without retraining: e.g. a TensorFlow-Lite approach on Zynq achieved **94% reduction in power-delay product (PDP)** and **420% DSP efficiency** for ResNet50 (INT8 vs FP32) ⁴. The same study showed INT8 VGG16 hit 250 FPS with 52% lower PDP ¹². Mixed-precision (e.g. 4-bit weights, 8-bit activations) can further save resources.

Pruning (weight elimination) sparsifies the model. On FPGAs, static sparsity must still be mapped carefully. Pruning 50% of filters or channels has been shown to cut operations by $>50\%$ and significantly lower energy ¹³. However, irregular sparsity complicates hardware: frameworks like MCBS mitigate this by grouping sparse bits (achieving $\sim 5.4\times$ DSP efficiency) ¹⁴ ¹⁰. Other works exploit zero-skipping (not performing MACs when inputs/weights are zero) to save energy ¹⁵ ¹⁶.

Recent FPGAs also support mixed quantization-aware pipelines. Some designs integrate binary or ternary CNN cores (FINN ⁷) for ultra-low-bit inference. Overall, quantization and pruning trade off accuracy for major energy savings. Empirical results often target $<2\%$ top-1 accuracy loss while doubling GOPS/W ⁴ ¹⁰.

Dataflow and Memory Reuse

Memory access is the dominant energy cost in CNNs ⁶. Systolic dataflows exploit data reuse to cut DRAM transfers. Common flows include **weight-stationary (WS)**, **output-stationary (OS)**, and **row-stationary (RS)**. WS keeps filter weights in PE registers and streams inputs; OS keeps partial sums local; RS tries to reuse rows of both inputs and weights. For example, a row-stationary SA in Google's TPU/ORCA architectures maximizes reuse in CNNs.

Hybrid or dynamic flows are emerging. Sestito's TrIM is effectively a novel WS variant: each weight stays in a PE while inputs sweep triangularly, ensuring every MAC is used and minimising idle time ³. Other architectures (FINN-R, FlightLLM) also show combining OS and WS strategies for different layers ¹⁷ ¹⁸. Automated mapping tools (e.g. fpgaConvNet ¹⁹) can even alternate dataflows per layer to optimize performance/energy.

On-chip memory architecture is critical. Modern FPGAs have multi-tiered memory: registers/LUTRAM (fast, tiny), Block RAM (e.g. 18–36Kb per block), UltraRAM (288Kb blocks on UltraScale+), and possibly HBM on advanced boards. Double-buffering (ping-pong) is used so computation can overlap with DMA transfers ²⁰ ²¹. Multi-level tiling (e.g. outer tiles in HBM, mid-level in URAM/BRAM, inner in registers) is a best practice ²¹. Efficient use of URAM (e.g. cascading 288Kb blocks ²²) and embedded registers allows large activation/weight buffers, cutting off-chip power.

Example Architectures and Metrics

Key FPGA CNN designs demonstrate both high throughput and efficiency. For instance, the Triangular SA achieved 453.6 GOPS on VGG16 at much lower energy than alternatives ³. The Versal DPU (Deep Learning Processor Unit, an IP accelerator on Xilinx Versal) delivers several TOPS (INT8) with pipelined MAC cores, although its details are proprietary. Finn accelerators achieve 50+ TOPS/W by binarized nets ²³ ²⁴. Overall, **TOPS/W** or **GOPS/W** (throughput per watt) is a key metric: many high-efficiency FPGA designs report tens to hundreds of GOPS/W ⁵ ²⁴. We also emphasize **energy per inference (Joule/inference)** and **accuracy vs energy trade-off** in evaluation.

Table 2 summarizes the FPGA case studies of [29], illustrating the range of designs and their efficiency:

Accelerator (Case Study)	Platform (FPGA/ Process)	Precision	Key Feature	Throughput (TOPS)	Utilization	Energy Eff.
FINN (binary nets) ²³	Zynq-7020 (28nm)	1b/1b (binary)	Streaming WS, layer- pipelined	~5 TOPS (binary ops)	90% LUT (no DSP)	–
FINN-R (quantized nets) ²⁵	Alveo U250 (16nm)	2–8b (QNN)	Automated tiling, partial WS	~50 TOPS (1- bit)	75% LUT, 0 DSP	~100 GOPS/ W
FlightLLM (sparse DSP) ¹⁷	Alveo U280 + HBM (16nm)	8b + sparsity	Sparse DSP chains (Skip zeros)	55 tokens/s (LLaMA-7B)	6345/9024 DSP	300– 400 GOPS/ W
SSR (Versal hybrid) ¹⁸	Versal VCK190 ACAP (7nm)	INT8	Spatial-seq. multi-engine	~26.7 TOPS	100% AIE, DSP	246– 453 GOPS/ W

These cases exemplify how dataflow (WS vs OS vs sparse) and precision choices drive performance and energy. Our design will draw on similar principles: maximize local reuse and minimize DRAM transfers to achieve high GOPS/W.

FPGA-Specific Considerations

Toolchains and Design Flow

Xilinx and Intel/AMD offer different development environments. Xilinx’s toolchain includes Vivado/Vitis and Vitis HLS for RTL/C++ design, as well as Vitis AI for AI accelerators (DPU IP) ²⁶. Intel provides Quartus Prime and the OpenCL/OneAPI flow for C/C++ design. In practice, we would use high-level synthesis (HLS) or RTL coding: for example, converting a C/C++ CNN implementation into an RTL accelerator via Vivado HLS ²⁶. After RTL generation, vendor tools (Vivado/Quartus) handle synthesis, placement, routing, and bitstream generation. Xilinx’s IP Integrator or Intel’s Platform Designer can integrate our accelerator with on-chip

CPUs (Zynq/SoC or Nios II) and memory controllers. For prototyping, frameworks like PYNQ (Xilinx Zynq boards) enable running Python/ONNX models on the FPGA ²⁶. Debugging/verification tools (ModelSim, Vivado Simulator) are used for functional checking.

FPGA Resources (DSP, BRAM, URAM, CLB)

Modern FPGAs have heterogeneous logic resources. For CNNs, DSP blocks are the workhorse for MACs. For example, Xilinx UltraScale+ FPGAs have up to ~3600 DSP slices, each 27×18b, enabling multiple 8-bit MACs per cycle. Block RAM (18–36 Kb per block) and UltraRAM (288 Kb per block on UltraScale+) provide on-chip storage ²². Intel FPGAs use M20K (20 Kb) and MLAB (memory in logic) blocks instead. Configurable logic (CLBs/ALMs) implement control, small memories, and LUT-based logic. We must budget resources: e.g. an 8×8 systolic array using INT8 might consume ~64–128 DSPs plus ~tens of BRAMs for buffers. Larger arrays (16×16) scale roughly quadratically in DSP use. LUT/FF utilization arises from address generators, control FSMs, and any LUT-MACs.

Floorplanning can improve timing and power: grouping DSPs and RAMs physically can shorten interconnect. Xilinx Vivado allows area/routing constraints to lock modules in columns or rows. Clocking is another issue: Xilinx provides MMCM/PLL blocks for frequency synthesis, and Zynq MPSoCs even support DVFS via on-board PMICs ²⁷. Intel FPGAs have their own PLL/DCM blocks. We plan a base clock (e.g. 100–300 MHz) and consider DVFS if supported (e.g. Ultrascale+ can scale PL voltage to 0.65–0.85 V ²⁷).

Power Estimation (Static vs. Dynamic)

FPGA power has both static (leakage) and dynamic (switching) components. Static power can be substantial (especially in advanced processes); dynamic power depends on toggle activity, switching capacitance, and voltage/frequency. Vendor tools estimate both: Xilinx’s Power Estimator/XPower Analyzer and Intel’s Power Analyzer use utilization and toggle rates to compute power. For example, Xilinx UG440 notes that larger UltraScale+ devices have significant static draw. We will use these tools for early estimates. Empirically, power-reduction techniques like clock gating or sparsity mainly cut dynamic power. For final validation, we will measure on-board (e.g. with Xilinx PMBus readings or external power probes) to capture real static/dynamic power under load. Montgomerie-Corcoran *et al.* built a power model that predicted total FPGA power within ~93% accuracy ²⁸, illustrating that careful modeling during design space exploration is feasible.

Low-Power Techniques

Algorithmic (Model-Level)

- **Quantization:** Using 8-bit fixed-point (or lower) instead of 32-bit floats drastically reduces computation cost and memory bandwidth. PTQ can often be applied, but *quantization-aware training* (QAT) may be needed to recover accuracy. The literature reports >90% reduction in power-delay product for INT8 inference ⁴. Mixed precision (e.g. 4-bit weights, 8-bit activations) balances low bit-width with accuracy ¹⁶.
- **Pruning/Sparsity:** Removing unimportant weights (filter or channel pruning) can cut MACs by >50% ¹³. Irregular sparsity still saves energy by skipping zero-MACs, but hardware must support it (zero-skipping logic or bit-serial schemes ¹⁵). Bit-level sparsity (observed in 8-bit quantized networks) yields inherent 50% zeros, enabling further bit-level compression ¹⁰.

- **Approximate Computing:** Techniques like weight/activation clipping or approximate multipliers can reduce switching. Eyeriss-like design mining shows memory traffic is highest power; selectively skipping or approximating infrequent computations can cut energy ⁶. For extreme low-power, binary/ternary networks eliminate multipliers entirely (FINN achieves 50+ TOPS/W) ²³.
- **Early Exit/Early Sizing:** Dynamically terminating inference when confidence is high or resizing data (sketching) can save cycles; however, this is out of scope for fixed-systolic designs.

Architecture-Level

- **Dataflow Reuse:** Maximizing reuse of fetched data via systolic flow drastically cuts DRAM accesses (the largest power draw ⁶). Triangular or row-stationary flows ensure high utilization and local accumulation ³ ⁶.
- **Memory Hierarchy:** Using register files, BRAM/URAM effectively (double buffering) saves energy. For example, an FPGA CNN with a multi-level memory (RF + SRAM) saw 11× speedup with reuse ²⁹. Turning off (clock-gating) idle memories or using URAM cascading for large on-chip buffers also reduces power ²².
- **Sparsity Skipping:** Architectures that detect zeros and skip MACs (sparse DSP chains ¹⁷) cut dynamic power. Flying zero-skipping, as in Eyeriss or recent DNN accelerators, disables entire MAC units on zeros.
- **DVFS:** If hardware supports it (e.g. Zynq MPSoC's PMIC ²⁷), lowering the PL voltage for non-critical workloads yields quadratic power savings. Likewise, clock frequency can be scaled down (using MMCM/DCM or on-the-fly switch) when throughput headroom exists. One FPGA study scaled clocks from 20–400 MHz and voltages from 0.65–0.85 V, enabling fine-grained DVFS ²⁷.
- **Power Gating:** Some FPGAs allow partial reconfiguration or gating of I/O banks. While full power gating of logic blocks is limited, unused FPGA columns (DSP/BRAM) can be clock-gated. Xilinx UltraScale+ allows ULTRARAM column sleep modes to reduce idle leakage.

RTL-Level

- **Clock Gating:** Explicit gating of clocks to registers/BRAM cuts dynamic power. Gundrapally *et al.* applied *Enhanced Clock Gating (ECG)* to MAC units and *Local Explicit Clock Gating/Enable (LECG/LECE)* on BRAMs ⁸ ³⁰. Their ResNet-20 accelerator on PYNQ-Z2 achieved ~22% power savings with ECG/LECG/LECE ⁸. Such gating inserts gating logic (XOR/AND latch) to disable clocks when outputs are unchanged ³¹ ³².
- **Operand/Path Gating:** If a multiplier's input is zero, disable its operation. FPGA designs can use a small logic check to skip the multiplier entirely (no DSP usage) when either operand is zero or negligible. This complements weight/activation sparsity, preventing switching in MAC paths.
- **Pipelining and Parallelism:** Proper pipelining (II=1) maximizes throughput but can increase register count (thus leakage) and toggle. One can balance by partially unrolling loops or gating pipeline stages when idle.
- **Dual/Ripple Buffers:** Double-buffering data fetch hides memory latency but toggles extra BRAM pages. Clock gating these buffers when idle can further save power.

Physical-Level

- **Voltage/Frequency Scaling:** As noted, fabric supplies can be trimmed. Though FPGAs typically use fixed rails, some (e.g. Zynq-UltraScale+) allow limited DVFS ²⁷. Lowering frequency also cuts dynamic power ($\sim f \cdot V^2$) at the expense of throughput.

- **Low-Power Libraries:** While ASICs have low-power cells, FPGAs rely on vendor internal libraries. Ensuring placement uses “low-leakage” resources (if selectable) is a consideration.
- **Thermal/PWM:** At board level, running FPGA at lower junction temperature or cycling idle periods (as in Idle-Waiting strategies ³³) can extend lifetime and reduce average power.
- **Integration:** Utilizing HBM (on Versal/Alveo) greatly boosts bandwidth, allowing lower-clock operation. Near-threshold computing is a research possibility if FPGAs ever support it natively.

In summary, an effective low-power CNN FPGA design combines *quantized low-precision inference, sparsity-aware dataflows, on-chip memory reuse, and clock/voltage gating*. Existing works show these techniques can multiply energy efficiency (e.g. 10–100× GOPS/W) compared to baseline floating-point implementations ⁴.

10 .

Design Space Exploration Methodology and Metrics

Design-space exploration (DSE) systematically evaluates trade-offs. We adopt a multi-objective DSE flow: vary architecture parameters (array size, dataflow, bitwidth, buffer sizes) and measure metrics including:

- **Throughput and Latency:** Inferences/sec or GOPS, and end-to-end latency per image.
- **Energy Efficiency (GOPS/W):** Throughput divided by measured power. High-end designs like FINN report ~100 GOPS/W ²⁴. We will target tens of GOPS/W.
- **Energy per Inference (J):** (Power × inference time). Critical for battery-powered/thermal budgets.
- **Resource Utilization:** % of available DSPs, BRAM, URAM, CLBs used. Ensures design fits the target FPGA.
- **Accuracy Loss:** Top-1 accuracy drop due to quantization/pruning. Maintain within 1–2% of floating-point.

Tools for DSE: high-level analytical models (e.g. roofline or fpgaConvNet graph models ¹⁹) can predict performance given unroll factors and memory bandwidth. We will also perform actual synthesis for key points to get accurate timing/power. For power modeling, the approach of Montgomerie-Corcoran *et al.* achieves ~93% accuracy and can drive power-focused DSE ²⁸. In practice, we will sweep parameters such as:

- **Array dimensions** (e.g. 8×8, 12×12, 16×16 PEs).
- **Data precision** (e.g. 8-bit, 4-bit, mixed).
- **Clock frequency** (e.g. 100–400 MHz).
- **Voltage** (if DVFS available, e.g. 0.7–1.0 V).
- **Tile size/block size** for input/output in memory tiling.
- **Pruning ratio** (0–90%).

For each setting, we record throughput (FPS), power (from XPower or board), energy/inference, and accuracy (from inference on a validation set). Pareto fronts (throughput vs. energy, accuracy vs. energy) will guide design choices.

Proposed Systolic-Array Microarchitecture

Based on the survey, we recommend a **weight-stationary systolic-array** microarchitecture with a Triangular Input dataflow, as it combines high data reuse with minimal off-chip traffic ³ ⁷ . The accelerator consists of:

- **PE Array:** A 2D grid of PEs. Each PE contains a multiplier, adder/accumulator, and local registers for one weight and one partial sum. PEs pass activations rightward and partial sums downward. In the weight-stationary flow, filter weights are loaded into PEs and held constant for a convolution window, reusing them for all input activations. Triangular Input Movement means input tiles enter from one edge and propagate diagonally, ensuring each PE receives appropriate input pixels ³ . This maximizes weight reuse in each PE while streaming inputs efficiently.
- **PE Design:** Each PE's datapath is a pipelined MAC: an activation register, a weight register, a multiplier, and an adder. After multiply and accumulate, the partial sum register feeds the next cycle or passes downward. For INT8 operations, we map the multiplier to a DSP slice (optionally packing two 8b MACs per DSP). 16-bit partial sums may use DSP adders or LUT-adders. Figure 1(a) (below) shows the PE's logic.

flowchart LR

```
Wreg[Weight Reg] --> Mult[MULT]
Areg[Act. Reg] --> Mult
Mult --> Add[ADD]
Acc[Accum Reg] <--- Add
```

- **Memory Interface:** Off-chip DRAM (DDR) feeds an on-chip buffer layer. A DMA engine reads input activations and weights into **BRAM/URAM** tiles. Double-buffering is used: while the systolic array processes one tile, the next tile is loaded. For example, input feature tiles and corresponding weight tiles are alternately ping-ponged between DRAM and on-chip block RAM. Large on-chip RAM (UltraRAM) may store entire weight matrices for small networks ²² . The control unit configures the sequence of tiles and issues load/computation commands.
- **Dataflow and Scheduling:** The overall convolution is tiled by output channels, input channels, and spatial windows. For each output tile, the control iterates over weight and input sub-tiles: weights are broadcast or preloaded into PEs; input activations are streamed in triangular fashion. Partial sums are accumulated across input channels. Once a full convolution window is done, the result is written back to output buffers (which in turn spill to DRAM if needed). This loop ordering (feature map stationary in width/height, weight-stationary in filter) provides high reuse ³ ⁶ .
- **Quantization Strategy:** We adopt 8-bit fixed-point for weights and activations (INT8), with symmetric or asymmetric scaling. Simulation studies (using CIFAR/ImageNet models) will determine if some layers need 4-bit or mixed precision. Initial weights are quantized and we apply quantization-aware retraining to recover accuracy. The PEs perform 8-bit MACs, fitting comfortably in DSP slices. If higher precision is needed (16-bit partial sums), PEs can cascade or use DSP adders.

- **Control and Scheduler:** A small FSM handles tile addresses and loop counts. The control logic also implements clock/operand gating: e.g. if a weight or activation is zero, that PE's clock can be gated (see RTL techniques below). Furthermore, when a tile completes, idle PEs are clock-gated to save dynamic power ⁸.

The block diagram in Figure 1(b) illustrates the system architecture:

```

flowchart LR
    DDR[External DRAM] -->|Load tiles| BRAM[On-chip Buffers<br/>(BRAM/URAM)]
    BRAM -->|Stream| SA[Systolic Array<br/>(PE Grid)]
    SA -->|Accumulate| ACC[Output Accumulator]
    ACC -->|Write-out| Out[Output Feature Maps]
    Ctrl[Control Unit] -.-> SA
    Ctrl -.-> BRAM

```

Figure 1: (a) PE datapath (multiplier + adder + regs). (b) Overall systolic-array accelerator architecture.

This architecture is scalable: by parameterizing the array size and buffer depths, it can target any FPGA. It embodies best practices (data reuse, pipelining, gating) for low power.

Implementation Plan

- **Target Platform:** The FPGA family is unspecified, but we assume a mid-to-high-end device (e.g. Xilinx UltraScale+ or Intel Stratix/Agilex). Our RTL should be generic enough to scale down.
- **Synthesis and Place-&Route:** We will implement the design in Vivado (Xilinx) or Quartus (Intel). High-level synthesis (HLS) can generate the PEs and data path from C/C++ prototypes, with pragmas for pipelining and unrolling. Careful constraints (timing exceptions, area grouping) will be used to meet clock targets (e.g. 200–300 MHz). Floorplanning may lock DSP columns or memory regions to reduce routing delays. We will pipeline through FFs to achieve II=1 per MAC.
- **Verification and Testbench:** We will write a comprehensive RTL testbench (SystemVerilog/VHDL) that feeds known image data through the accelerator and checks output against a golden model (e.g. TensorFlow reference). Functional simulation (modelsim/Vivado sim) will ensure correctness. We will also use hardware-in-the-loop testing on an FPGA dev board (e.g. running PYNQ/Zynq) to run sample CNN inference and compare to software reference. Formal equivalence checking (if available) can verify that HLS output matches the algorithmic spec.
- **Clocking and Power:** The design uses a global clock; clock gating cells will be inserted around idle modules (MACs, BRAMs) in the RTL. We will generate a single clock domain or a few domains for different array sections. For power estimation, we will run Vivado/Quartus power analysis after place-and-route, feeding in toggle activity from gate-level simulation. To measure actual power, we plan on-board measurements: e.g. using an FPGA's built-in PMBus interface or external power meters to record Vccint and Vccbram currents. This real measurement will validate (or calibrate) the vendor tool estimates.

- **Design Iteration:** Based on DSE results, we may iterate the architecture. For example, if memory bandwidth is insufficient, we might increase on-chip tile reuse or reduce array size. If timing closure fails, we may cut pipeline depth or lower clock speed (trading throughput for frequency).

Benchmarks and Datasets

For evaluation, we will use standard image classification benchmarks: **CIFAR-10/100** (small 32×32 images, 10/100 classes) for quick development and **ImageNet** (or a representative subset like Tiny-ImageNet) for large-scale testing. Typical network models include ResNet-20/50, VGG-16, and MobileNet. These are widely used in FPGA literature ³⁴ ³⁵. We will train (or obtain pretrained) models in PyTorch/TensorFlow, then apply INT8 quantization-aware training to maintain accuracy. If pruning is used, we will fine-tune the sparse model to recover performance. The ONNX format and tools like TensorFlow Lite can generate quantized models.

Datasets beyond ImageNet: CIFAR-10/100 ensures we can measure ~95%+ baseline accuracy and see small drops after quantization ³⁵. We may also include the **CNN object-detection SkyNet** benchmark (UAV imagery) since it is used in DVFS studies ³⁶. Metrics to report include top-1 accuracy (baseline vs quant/prune), throughput (images/sec on FPGA), and real power consumption during inference.

Expected Results and Evaluation Templates

We expect our low-power systolic design to achieve competitive throughput with significantly reduced energy per inference. For example, prior work saw ~68% latency reduction and >50% PDP reduction with INT8 quantization on VGG16 ¹². Our goal is similar: e.g. 2×–3× increase in fps or GOPS at constant accuracy, or 50–70% lower energy per inference. Energy efficiency (GOPS/W) on the order of 30–100 GOPS/W is realistic ⁵ ²⁴. Accuracy loss should be ≤1–2% top-1 after quantization/pruning.

Sample Evaluation Table: We propose to collect results in tables like below (filled with hypothetical/example values):

Design Variant	Precision	Freq (MHz)	Throughput (FPS)	Power (W)	Energy (J/ img)	Accuracy (%)
Baseline (INT8)	INT8	200	150	4.0	0.0267	93.5
+Clock Gating	INT8	200	150	3.2	0.0213	93.5
+Weight Pruning	INT8+50% pr.	200	180	3.2	0.0178	92.8
+Mixed Precision	4b/8b	200	300	3.6	0.0120	92.0
Proposed SA (TrIM)	INT8	250	453.6 (GOPS)	5.0	0.0110	–

(Note: Values above are illustrative. Actual results would be inserted.)

Metrics like TOPS/W and energy per inference will be tabulated. We will compare against literature numbers (Table 1 references) to contextualize our efficiency.

Experiments and Parameter Sweeps

To explore the design space, key parameters and ranges include:

- **Array size:** e.g. 4×4, 8×8, 12×12, 16×16 PEs.
- **Data precision:** INT8 vs mixed 4/8-bit vs 1-bit (extreme).
- **Clock frequency:** 100–300 MHz (subject to timing).
- **Supply voltage:** 0.7–1.0 V (if DVFS supported) ²⁷.
- **Off-chip bandwidth:** (simulate scenarios with different DDR speeds).
- **Tile dimensions:** e.g. 16×16 vs 32×32 pixel tiles.
- **Sparsity levels:** 0%–90% zeros in weights or activations.
For each combination, we will simulate performance and power (via FPGA tools) and measure accuracy. A regression or Pareto analysis will identify optimal points (e.g. maximum throughput under a power budget).

Suggested experiments:

1. **Precision sweep:** Run ResNet-20 on CIFAR-10 at INT8, INT4, binarized. Measure accuracy and GOPS/W.
2. **Array scaling:** Fix INT8, vary PE count. Observe scaling of throughput vs resource use.
3. **DVFS test:** For a fixed design, vary frequency (and voltage) to map the performance-energy curve.
4. **Clock gating effect:** Compare same design with/without ECG/LECG on power. Gundrapally *et al.* saw ~22% power drop ⁸.
5. **Pruning ratio:** Prune ResNet to various sparsity levels, retrain, measure MAC reduction and energy savings.
6. **Dataflow variant:** (If feasible) Implement a small output-stationary version and compare to weight-stationary.

This systematic sweep will validate the low-power strategies and identify the best operating point for our target application (e.g. highest frames/s per watt).

Risks, Limitations, and Future Work

- **Accuracy vs. Compression:** Aggressive quantization/pruning may degrade accuracy. Ensuring <2% loss may require careful retraining, which is time-consuming.
- **FPGA Support:** DVFS and power gating support vary by board. Some boards (low-cost) cannot change core voltage dynamically. Our design may assume features (like Xilinx ZCU104 DVFS ²⁷) that are not universal.
- **Tool Limitations:** HLS can simplify design but may not meet timing at high frequencies. A fallback to hand-written RTL (as in [47]) could be needed for ultimate performance.
- **Resource Constraints:** If the target FPGA is smaller, large arrays or high-bandwidth buffers may not fit. The design must scale gracefully or gracefully degrade (e.g. fewer PEs with more reuse).
- **Verification Complexity:** Exhaustively testing the accelerator on all CNN layers is challenging. Mismatches in data ordering or quantization can introduce subtle bugs. Robust testbenches and possibly emulation (FPGA-in-the-loop) may be required.

- **Environmental Power Variation:** Board-level power measurements can vary with temperature and peripheral loads. Ensuring consistent conditions is important for fair energy comparisons.

Future extensions could include multi-FPGA scaling, integration with on-chip AI engines (Versal ACAP's AIEs), or exploring emerging memory (RRAM) for deeper on-chip buffering. Also, machine-learning-based DSE or online dynamic adaptation (e.g. switching flows per layer) are interesting directions.

```

gantt
  title Proposed Thesis Timeline
  dateFormat YYYY-MM-DD
  Literature Survey      :a1, 2026-04-14, 60d
  Design & DSE          :a2, after a1, 30d
  Microarchitecture Design: a3, after a2, 45d
  RTL/HLS Implementation :a4, after a3, 90d
  FPGA PnR & Timing      :a5, after a4, 60d
  Verification & Testing  :a6, 2026-09-01, 120d
  Power/Performance Eval :a7, after a6, 60d
  Results Analysis      :a8, after a7, 30d
  Thesis Writing        :a9, after a8, 60d

```

Figure 2: Proposed timeline for thesis tasks (Gantt chart).

Table 3: Low-power Techniques (examples)

Technique	Level	Benefit	References
Reduced Precision (INT8/4-bit)	Algorithmic	Fewer bits → lower MAC cost & memory bandwidth ⁴	[4],[35]
Pruning (50% weights)	Algorithmic	~50% fewer MACs → lower dynamic power ¹³	[20],[35]
Clock Gating (ECG/LECG/LECE)	RTL/Logic	Shuts off idle cycles/units → ~20% dynamic power ↓ ^{8 30}	[47],[46]
Voltage/Freq Scaling (DVFS)	Architecture	Lower V/I for slower clocks → V ² power savings ²⁷	[24]
Dataflow/Reuse (WS/TrIM)	Architecture	Maximizes on-chip reuse → fewer DRAM transfers ^{3 6}	[10],[35]
On-chip Memory Hierarchy (BRAM/URAM)	Architecture	Multi-level caches reduce off-chip access energy	[20],[32]
Sparsity Exploitation (Zero-skipping)	Architecture	Skip zero MACs → lower switching energy	[27],[20]

Each of these techniques will be applied and evaluated in our design. For instance, combining INT8 quantization with clock gating and a data-reuse flow is expected to yield multiplicative energy savings ⁸ ³ .

Table 4: FPGA Resource Comparison (selected examples)

FPGA Family / Device	DSP Slices	BRAM / M20K	URAM	Node (nm)
Xilinx Zynq-7000 (e.g. ZU7)	~220	~20 Mb	0	28/16
Xilinx UltraScale+ (e.g. VU9P)	2880	22.9 Mb	252 Mb ²²	16
Intel Arria 10 GX	1518	65.4 Mb	0	20
Intel Stratix 10 (SX)	5574	52.9 Mb	0	14
Xilinx Versal VCK190	~3600+	~196 Mb	~218 Mb	7

(Values approximate; URAM only in Xilinx UltraScale+ and Versal). These resources guide our design sizing: e.g., a 16×16 INT8 SA uses ~256 DSPs (9% of VU9P) and several Mb of RAM, fitting comfortably in high-end devices.

Table 5: Key CNN Accelerator Papers

Paper (Ref)	Year	Arch. / Flow	FPGA / Node	CNN Model	Precision	Peak Throughput	Energy Saving
Wei <i>et al.</i> ⁹	2017	Systolic Array (WS)	FPGA (28nm)	AlexNet/ CNN	FP32/ INT8	461 GFLOPS FP, 1.2 TOPS (8-bit)	–
Du <i>et al.</i> ⁵	2022	Systolic (configurable)	Zynq ZC706 (28nm)	VGG16	INT8	115.2 GOPS	30.32 GOPS/W ⁵
Sestito <i>et al.</i> ³	2024	Triangular SA (WS)	FPGA (16nm)	VGG16/ AlexNet	INT8	453.6 GOPS	~11.9× vs prior
Hu <i>et al.</i> ¹⁰	2025	Sparse SA (MCBS)	Zynq/ FPGA (unspec)	ResNet50/ VGG16	4/8-bit + BLSQ	–	8.87× (vs SOTA)
Gundrapally <i>et al.</i> ⁸	2024	Systolic (Tensil)	PYNQ-Z2 (28nm)	ResNet-20 (CIFAR-10)	INT8	–	22% power ↓ (ResNet-20)

These tables and discussions together outline the **design space** and expectations. Our work will position itself among these by achieving high throughput with aggressive power savings.

Sources: We prioritize recent conferences/journals and vendor guides. Notable sources include Qian *et al.* (JSA 2025) survey ¹ , vendor manuals (Xilinx UG440 ²²), and hardware papers ³ ⁵ ⁸ . All cited facts above are drawn from these primary sources.

¹ FPGA-based Acceleration for Convolutional Neural Networks: A Comprehensive Review

<https://arxiv.org/html/2505.13461v1>

² ⁵ ¹³ ¹⁶ ²⁹ An Energy-Efficient Convolutional Neural Network Processor Architecture Based on a Systolic Array

<https://www.mdpi.com/2076-3417/12/24/12633>

³ arxiv.org

<https://arxiv.org/pdf/2408.10243>

⁴ ¹² ³⁴ Post-training quantization for efficient FPGA-based neural network acceleration - ScienceDirect

<https://www.sciencedirect.com/science/article/abs/pii/S0167926025001658>

⁶ ¹¹ ¹⁹ ²⁸ Power-Aware FPGA Mapping of Convolutional Neural Networks

[https://steliosven10.github.io/papers/\[2019\]_fpt_power_aware_fpga_mapping_of_cnns.pdf](https://steliosven10.github.io/papers/[2019]_fpt_power_aware_fpga_mapping_of_cnns.pdf)

⁷ ¹⁷ ¹⁸ ²⁰ ²¹ ²³ ²⁴ ²⁵ Dataflow & Tiling Strategies in Edge-AI FPGA Accelerators: A Comprehensive Literature Review

<https://arxiv.org/html/2505.08992v3>

⁸ ²⁶ ³⁰ ³¹ ³² ³⁵ A High-Performance and Ultra-Low-Power Accelerator Design for Advanced Deep Learning Algorithms on an FPGA

<https://www.mdpi.com/2079-9292/13/13/2676>

⁹ (PDF) Automated Systolic Array Architecture Synthesis for High Throughput CNN Inference on FPGAs

[https://www.researchgate.net/publication/](https://www.researchgate.net/publication/317574806_Automated_Systolic_Array_Architecture_Synthesis_for_High_Throughput_CNN_Inference_on_FPGAs)

[317574806_Automated_Systolic_Array_Architecture_Synthesis_for_High_Throughput_CNN_Inference_on_FPGAs](https://www.researchgate.net/publication/317574806_Automated_Systolic_Array_Architecture_Synthesis_for_High_Throughput_CNN_Inference_on_FPGAs)

¹⁰ ¹⁴ ¹⁵ An FPGA-based bit-level weight sparsity and mixed-bit accelerator for neural networks - ScienceDirect

<https://www.sciencedirect.com/science/article/abs/pii/S1383762125001353>

²² UltraRAM support for UltraScale+ Devices - UltraRAM support for UltraScale+ Devices - 2025.2 English - UG440

<https://docs.amd.com/r/en-US/ug440-xilinx-power-estimator/UltraRAM-support-for-UltraScale-Devices>

²⁷ ³⁶ research.nottingham.edu.cn

[https://research.nottingham.edu.cn/files/31438821/](https://research.nottingham.edu.cn/files/31438821/Optimizing_energy_efficiency_of_CNN_based_object_detection_with_dynamic_voltage_and_frequency_scaling.pdf)

[Optimizing_energy_efficiency_of_CNN_based_object_detection_with_dynamic_voltage_and_frequency_scaling.pdf](https://research.nottingham.edu.cn/files/31438821/Optimizing_energy_efficiency_of_CNN_based_object_detection_with_dynamic_voltage_and_frequency_scaling.pdf)

³³ Configuration-aware approaches for enhancing energy efficiency in FPGA-based deep learning accelerators - ScienceDirect

<https://www.sciencedirect.com/science/article/pii/S1383762125000827>